# Apuntes · Sesión 2 · Promesas

Oct 29, 2026

En la sesión 1 viste que, con callbacks, los pasos asíncronos que dependen unos de otros forman una pirámide. Las promesas resuelven ese problema: el código queda plano y los errores se gestionan en un solo sitio.

## 1. Qué es una promesa

Una **promesa** es un objeto que representa un resultado que todavía no existe, pero que llegará o fallará más adelante.

Con callbacks, la función recibía «lo que hay que hacer cuando termine». Con promesas, la función termina enseguida y te **devuelve** la promesa; después tú decides qué hacer con el resultado.

**La analogía del tique.** En un restaurante de comida rápida pides y te dan un tique. Todavía no tienes la comida, pero tienes algo que la representa. Mientras esperas puedes hacer otras cosas. Al final, una de dos: te dan la comida (la promesa se **cumple**) o te dicen que se ha acabado (la promesa se **rechaza**).

| Estado | Significado | En el restaurante |
|---|---|---|
| `pending` (pendiente) | Aún no hay resultado | Esperando con el tique |
| `fulfilled` (cumplida) | Terminó bien y tiene un valor | Te dan la hamburguesa |
| `rejected` (rechazada) | Falló y tiene un error | «No quedan hamburguesas» |

Una promesa empieza en `pending` y pasa **una sola vez** a `fulfilled` o a `rejected`. Después ya no cambia nunca.

## 2. Crear y consumir una promesa

### Fabricar: siempre la misma plantilla

Una pizzería tarda 2 segundos en hacer una pizza. Si la piden con piña, no la hace:

```
function pedirPizza(tipo) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (tipo === 'piña') reject(new Error('Aquí no hacemos pizza con
piña'));
      else resolve('Pizza ' + tipo);
    }, 2000);
  });
}
```

- La primera línea (`return new Promise((resolve, reject) => {`) y el cierre (`});`) son **siempre iguales**: es una plantilla que se copia. Solo cambia lo de dentro.
- `new Promise` te da dos «timbres», `resolve` y `reject`. Cuando el trabajo termina, tocas uno de los dos:
  - `resolve(valor)`: «ha ido bien y este es el resultado». La promesa pasa a `fulfilled`.
  - `reject(error)`: «ha ido mal y este es el motivo». La promesa pasa a `rejected`.
- Se tocan **dentro** del `setTimeout`, porque solo ahí ha pasado el tiempo y se sabe cómo ha ido.
- `resolve` es un parámetro que recibe una función, igual que `fn` en `repetir` (sesión 1). La diferencia es que esta función no la pones tú: te la da JavaScript al crear la promesa.

Rechaza siempre con `new Error('mensaje')`, no con un texto suelto: así el error trae su mensaje en `error.message`.

## Usar: then, catch y finally

```
pedirPizza('margarita')
  .then(pizza => console.log('Ha llegado:', pizza))
  .catch(error => console.log('Problema:', error.message))
  .finally(() => console.log('Fin del pedido'));
// a los 2 s:
// Ha llegado: Pizza margarita
// Fin del pedido
```

| Método | Cuándo se ejecuta su callback | Qué recibe |
|---|---|---|
| `.then(fn)` | Si la promesa se cumple | Lo que se pasó a `resolve` |
| `.catch(fn)` | Si la promesa se rechaza | Lo que se pasó a `reject` |
| `.finally(fn)` | Siempre, vaya bien o mal | Nada |

Prueba a cambiar `'margarita'` por `'piña'`: sale `Problema: Aquí no hacemos pizza con piña` y, después, `Fin del pedido`.

**Lo que devuelve no es la pizza.** `pedirPizza` termina enseguida, antes de que la pizza exista. Lo que devuelve es la promesa, el tique:

```
const tique = pedirPizza('barbacoa');
console.log(tique);                                    // Promise {
<pending> }
tique.then(pizza => console.log('Ha llegado:', pizza));  // a los 2 s: Ha
llegado: Pizza barbacoa
```

La pizza solo se recoge con `then`. Y a `then` se le pasa una **función**, no el resultado de ejecutarla: `.then(console.log('Ha llegado'))` se ejecutaría ya, sin esperar.

> **Ahora te toca: ejercicios 2.1, 2.2 y 2.3.** En `alumnado/sesion2/ejercicios.js`: usar `hervirAgua` (2.1), crear `lanzarMoneda` (2.2) y crear `comprobarEdad` (2.3). Para los dos últimos, copia la plantilla de `pedirPizza` y cambia lo de dentro. Enunciados en la hoja de ejercicios.

## 3. Encadenar promesas

Esta es la parte más importante de la sesión. Se basa en tres reglas.

**Regla 1: `then` siempre devuelve una promesa nueva.** Por eso se puede poner un `.then` detrás de otro.

**Regla 2: lo que devuelves en un `then` lo recibe el siguiente `then`.**

```
pedirPizza('margarita')
  .then(pizza => pizza + ' con extra de queso')   // recibe 'Pizza margarita'
y devuelve algo nuevo
  .then(pedido => console.log(pedido));          // recibe lo que devolvió el
anterior
// a los 2 s: Pizza margarita con extra de queso
```

Si lo que devuelves es **otra promesa**, el siguiente `then` espera a que esa promesa se cumpla y recibe su valor. Esto es lo que permite encadenar pasos asíncronos uno detrás de otro.

**Regla 3: si algo falla en cualquier punto, se saltan todos los `then` y se va directo al `catch`.**

## Un ejemplo con tres pasos

La misma pizzería, ahora con tres pasos. Para no esperar tanto, cada paso tarda 1 segundo; todos devuelven una promesa y cada uno necesita el resultado del anterior. Son las mismas `pedirPizza` y `hornearPizza` que tienes ya hechas en `ejercicios.js`:

```
function pedirPizza(tipo) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (tipo === 'piña') reject(new Error('No hacemos pizza con piña'));
      else resolve('Pizza ' + tipo);
    }, 1000);
  });
}

function hornearPizza(pizza) {
  return new Promise(resolve => {
    setTimeout(() => resolve(pizza + ' horneada'), 1000);
  });
}

function llevarPizza(pizza) {
  return new Promise(resolve => {
    setTimeout(() => resolve(pizza + ' y entregada en casa'), 1000);
  });
}
```

Se encadenan así, un paso **debajo** de otro:

```
pedirPizza('barbacoa')
  .then(pizza => hornearPizza(pizza))
  .then(pizza => llevarPizza(pizza))
  .then(mensaje => console.log(mensaje))
  .catch(error => console.log('Problema:', error.message));
// a los 3 s: Pizza barbacoa horneada y entregada en casa
```

| Momento | Qué pasa | Dato que viaja |
|---|---|---|
| 1 s | Pizza pedida: el primer `then` la recibe y **devuelve** la promesa del horno | `'Pizza barbacoa'` |
| 2 s | Pizza horneada: el segundo `then` la recibe y devuelve la promesa del reparto | `'Pizza barbacoa horneada'` |

| Momento | Qué pasa | Dato que viaja |
|---|---|---|
| 3 s | Pizza entregada: el tercer `then` la imprime | `'Pizza barbacoa horneada y entregada en casa'` |

Con callbacks, esto sería una pirámide como la de la cuenta atrás de la sesión 1. Con promesas queda plano y con un solo `catch` para todos los errores.

**Si se pide con piña**, al primer segundo `pedirPizza` se rechaza. Por la regla 3, se saltan los tres `then` y se ejecuta el `catch`: `Problema: No hacemos pizza con piña`. El horno y el repartidor ni siquiera empiezan.

## Error típico 1: olvidar el return

Es la regla de las funciones flecha: **con llaves hay que escribir `return`**.

```
pedirPizza('barbacoa')
  .then(pizza => {
    llevarPizza(pizza);                    // sin return: el repartidor
sale, pero nadie lo espera
  })
  .then(mensaje => console.log(mensaje));   // al segundo 1: undefined
```

El primer `then` no devuelve la promesa del repartidor, así que devuelve `undefined`. El siguiente `then` no tiene nada que esperar y se ejecuta enseguida. Arreglo: `return llevarPizza(pizza);` o la flecha corta `pizza => llevarPizza(pizza)`.

## Error típico 2: meter un then dentro de otro

```
// Funciona, pero la pirámide ha vuelto
pedirPizza('barbacoa').then(pizza => {
  hornearPizza(pizza).then(horneada => {
    llevarPizza(horneada).then(mensaje => console.log(mensaje));
  });
});
```

La solución es **devolver** la promesa y poner el siguiente `then` fuera, al mismo nivel, como en el ejemplo de arriba.

> **Ahora te toca: ejercicio 2.4.** En `alumnado/sesion2/ejercicios.js` hay un código que debería mostrar `Pizza margarita horneada`, pero muestra `undefined`. Encuentra el fallo y corrígelo; la clave está en los errores típicos de este apartado. Enunciado en la hoja de ejercicios.

# 4. esperar(ms): una promesa que solo espera

`esperar` es la plantilla de `pedirPizza`, pero sin pizza: el trabajo que tarda es solo esperar.

```
function esperar(ms) {
  return new Promise(resolve => {
    setTimeout(() => resolve(), ms);
  });
}
```

- Cuando pasan los `ms` milisegundos, el `setTimeout` llama a `resolve()` y la promesa se cumple.
- `resolve()` va vacío porque no hay nada que entregar: solo avisa de que ya ha pasado el tiempo.
- No hay `reject` porque esperar nunca falla.

Se usa con `then`, como cualquier promesa:

```
console.log('Empiezo');
esperar(2000).then(() => console.log('Han pasado 2 segundos'));
```

Y como devuelve una promesa, se puede **encadenar** con otras esperas: si un `then` devuelve `esperar(1000)`, el siguiente `then` espera un segundo (regla 2). Así, la cuenta atrás de la sesión 1 se puede escribir sin pirámide.

# 5. Microtareas: las promesas tienen prioridad

En la sesión 1 vimos una sola cola de espera. En realidad hay dos:

| Cola | Qué entra en ella | Cuándo se atiende |
|---|---|---|
| Microtareas | Los callbacks de `then`, `catch` y `finally` | Primero: se vacía entera en cuanto termina el código normal |
| Tareas | `setTimeout`, clics y otros eventos | Después |

```
console.log('inicio');
setTimeout(() => console.log('temporizador'), 0);
Promise.resolve().then(() => console.log('promesa'));
console.log('fin');
```

Salida:

```
inicio
fin
promesa
temporizador
```

1. `inicio` y `fin` son código normal: salen primero, en orden.
2. Cuando el código normal termina, el event loop mira **primero** las microtareas: sale `promesa`.
3. Después mira la cola de tareas: sale `temporizador`, aunque su retardo fuera de 0 ms.

Lo importante: **el código normal siempre termina antes que cualquier callback**, y entre los callbacks que ya están listos, los de las promesas van antes que los de `setTimeout`.

## 6. Resumen y errores típicos

| Concepto | En una frase | Ejemplo |
|---|---|---|
| Promesa | Objeto que representa un resultado futuro | `new Promise((resolve, reject) => ...)` |
| `resolve` / `reject` | Funciones para cumplir o rechazar la promesa | `resolve('Pizza ' + tipo)`, `reject(new Error('...'))` |
| Estados | Pendiente, cumplida o rechazada; solo cambia una vez | `pending` → `fulfilled` / `rejected` |
| `then` / `catch` / `finally` | Qué hacer si va bien, si falla y siempre | `p.then(...).catch(...)` |
| Encadenar | Un `then` detrás de otro, devolviendo valores o promesas | `.then(pizza => hornearPizza(pizza))` |
| Promisificar | Envolver algo basado en callbacks en una promesa | `esperar(ms)` |
| Microtarea | Cola de los `then`, atendida antes que la de `setTimeout` | Por eso `promesa` sale antes que `temporizador` |

### Errores típicos

| Error | Qué pasa | Cómo se arregla |
| --- | --- | --- |
| `then` con llaves sin `return` | El siguiente `then` recibe `undefined` y no espera | Escribir `return`, o usar la flecha corta sin llaves |
| Meter un `then` dentro de otro | Vuelve la pirámide y se pierde el `catch` único | Devolver la promesa y encadenar al mismo nivel |
| Cadena sin `catch` | Aparece *Uncaught (in promise)* en la consola | Terminar cada cadena con `.catch(...)` |
| Rechazar con un texto suelto | No hay `err.message` | Rechazar con `new Error('mensaje')` |